FYS-STK4155: Project 2

Github repository: https://github.com/nillescorner/FYS-STK4155-Group/tree/main/Project_2

Pernille Xu Amundsen, Maria Andersen, Hannah Westgaard

(Dated: November 9, 2026)

Neural networks can learn their own features from raw data, but they are more costly than simpler methods. In this project, we implement a feed-forward network with backpropagation. We apply the network to regression on the one-dimensional Runge function and to classification of handwritten digits in the MNIST data set, and compare it with OLS, Ridge, Lasso and logistic regression. For the Runge function, a network with one hidden layer of 50 sigmoid nodes, trained with SGD and Adam, reached a test MSE of 0.013, which was the lowest error for different adaptive optimizers, which we got by using the learning rate $\gamma = 3.2 \cdot 10^{-3}$. For this smooth, one-dimensional function, one hidden layer with 50 nodes is sufficient, when using sigmoid, as larger networks gave no further improvement. ReLU and LeakyReLU did best at $\gamma = 0.01$ with test MSEs of 0.0157 and 0.0158. For these activation functions, two hidden layers with 50 nodes were sufficient. With ℓ_1 and ℓ_2 penalties ($\gamma = 10^{-3}$, and $\lambda = 10^{-4}$ for sigmoid and $\lambda = 10^{-3}$ for LeakyReLU) the networks reached test MSEs of 0.0147-0.0163 compared to 0.0219 and 0.0220 for Ridge and Lasso with parameters from Project 1. Neural networks were more costly to train, but gave lower MSE than Project 1 methods. The neural network built for Runge was applied to MNIST with minimal structural changes but different parameters. It is clear that the number of hidden layers and their nodes had the greatest effect. The best model achieved an accuracy of 98.1%, a significant increase from the baseline logistic regression of 93%, but at 11 times the computational cost.

I. INTRODUCTION

Neural networks are a machine learning method that is inspired by the human brain, in the way it finds patterns and makes predictions. They work by stacking artificial simple "neurons" in layers and learns to recognize patterns in order to make an educated guess, just as our brain does every day [1].

Neural networks are widely used in a range of different fields, and in this project we will investigate two different types of problems, regression and classification problems. A typical regression problem is that of house price predictions, which neural networks can be trained on in order to give accurate predictions based on property features of the house. An example of a classification problem could be image recognition, where an image of a fruit is sent in and the network then has to predict what fruit it is.

In project 1 [2], we trained a linear regression model, where we chose the features in the design matrix. For a neural network we instead send in the input data, and let the network itself learn its own features. The network does its learning in what is called hidden layers, that are in-between the input layer and the output layer. This makes the network far more flexible, since it can represent features we would not think to include ourselves, but it comes at a much higher computational cost and with many more choices to tune (chapter 8 in [3]). The central question of this project is therefore when a neural network is with the extra cost compared with simpler methods, such as linear and logistic regression.

In order to compare to the simpler linear model we made in Project 1, we will use the same one-dimensional Runge function on our neural network. This will be the regression problem we will test our network on, but we will also look at a classification problem, where we will be using the MNIST database [4]. This is a database containing 70 000 images of handwritten digits from 0 to 9, which we will use to train our network and then later see if it can predict the images correctly when tested.

We have, in this project, written our own feed-forward network with backpropagation, and verified that the gradients from our backpropagation agree with automatic differentiation. We then train the network using some of the different gradient methods we used in Project 1: plain GD, SGD, SGD with RMSprop and SGD with Adam. For the regression problem, we compare the network predictions with the predictions from the OLS method from Project 1. For our neural network, we study the effect of changing different parameters, such as the learning rate, the network size and what activation function is used, where we use sigmoid, ReLU and Leaky ReLU. We also add the ℓ_1 and ℓ_2 penalty terms, to study how these affect the predictions made. For the classification problem, we use a softmax output layer with cross entropy, and compare the network with logistic regression.

In section II the theory of the cost functions and their gradient, and activation functions are presented. In section III, the methods and implementation of the network is described. In section IV, the results are presented, as well as discussed. The code for this project is found in the GitHub repository.

II. THEORY

A. Cost functions and gradients

1. Mean Square Error (MSE)

We will need the expression for the cost function of the mean squared error (MSE), which is given as

$$C(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \tilde{y}_i)^2 = \frac{1}{n}||\boldsymbol{y} - \tilde{\boldsymbol{y}}||_2^2, \tag{A.1}$$

where $\boldsymbol{\theta}$ are the model parameters, n is number of data points, y_i is the target data point and $\tilde{y}$ is the model prediction. The gradient of this cost function is

$$\frac{\partial C}{\partial \tilde{y}_i} = \frac{2}{n}\left(\tilde{y}_i - y_i\right),$$

and with a linear model, $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$, so the gradient becomes

$$\nabla_{\boldsymbol{\theta}} C = \frac{2}{n}\boldsymbol{X}^T\left(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\right). \tag{A.2}$$

This cost function, and corresponding gradient is for ordinary least squares (OLS), as there is no penalty term. For Ridge, with a ℓ_2 norm, we have the cost function and corresponding gradient, given as

$$C_{L_2}(\boldsymbol{\theta}) = \frac{1}{n}||\boldsymbol{y} - \tilde{\boldsymbol{y}}||_2^2 + \lambda||\boldsymbol{\theta}||_2^2, \tag{A.3}$$

$$\nabla_{\boldsymbol{\theta}} C_{L_2} = \frac{2}{n}\boldsymbol{X}^T\left(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\right) + 2\lambda\boldsymbol{\theta}, \tag{A.4}$$

where λ is the penalty.
For Lasso, with a ℓ_1 norm, we have the cost function and corresponding gradient, given as

$$C_{L_1}(\boldsymbol{\theta}) = \frac{1}{n}||\boldsymbol{y} - \tilde{\boldsymbol{y}}||_2^2 + \lambda||\boldsymbol{\theta}||_1, \tag{A.5}$$

$$\nabla_{\boldsymbol{\theta}} C_{L_1} = \frac{2}{n}\boldsymbol{X}^T\left(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\right) + \lambda\ \mathrm{sgn}(\boldsymbol{\theta}), \tag{A.6}$$

where $|\theta_j|$ is not differentiable at 0, so we have the $\mathrm{sgn}(\boldsymbol{\theta})$ function which is given as

$$\mathrm{sgn}(\boldsymbol{\theta}) = \begin{cases} +1, & \theta_j > 0 \\ 0, & \theta_j = 0 \\ -1, & \theta_j < 0 \end{cases}$$

2. Binary cross entropy

The binary cross entropy is used for binary classification problems, and the cost function is given as (eq.5.13 in [3])

$$C(\boldsymbol{\theta}) = -\sum_{i=1}^{n}\left[y_i \log p_i + (1 - y_i)\log(1 - p_i)\right], \tag{A.7}$$

where $p_i = \sigma\left(\theta_0 + \theta_1 x_i\right)$ is the probability, and σ is the Sigmoid function (eq. 5.3 in [3]). The gradient of the cost function (eq. 5.17 in [3]), becomes

$$\nabla_{\boldsymbol{\theta}} C = -\boldsymbol{X}^T\left(\boldsymbol{y} - \boldsymbol{p}\right). \tag{A.8}$$

This is again the cost function without a penalty term. We can add the ℓ_1 and ℓ_2 norm terms again to get the expressions for Lasso and Ridge. We get

$$C_{L_2} = C + \lambda||\boldsymbol{\theta}||_2^2,\ \ \nabla_{\boldsymbol{\theta}} C_{L_2} = \nabla_{\boldsymbol{\theta}} C + 2\lambda\boldsymbol{\theta} \tag{A.9}$$

$$C_{L_1} = C + \lambda||\boldsymbol{\theta}||_1,\ \ \nabla_{\boldsymbol{\theta}} C_{L_1} = \nabla_{\boldsymbol{\theta}} C + \lambda\ \mathrm{sgn}(\boldsymbol{\theta}) \tag{A.10}$$

3. Multiclass cross entropy

For multiclass cross entropy, the targets are one-hot encoded, meaning that only the sample belonging to the class we are in will contribute. The softmax function will turn the values it gets into probabilities, in the following way

$$p_{ik} = \frac{\exp(z_{ik})}{\sum_{l=1}^{K}\exp(z_{il})},$$

and the cost function will be (eq. 5.28 in [3])

$$C = -\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log p_{ik}, \tag{A.11}$$

where K is the number of classes. To find the gradient of the cost function (eq. 5.29 in [3]) we first find the derivative of the softmax, then we find the derivative of the cost with respect to the probabilities, and lastly use the chain rule so that we in the end obtain

$$\nabla_{\boldsymbol{\theta}} C = -\boldsymbol{X}^T\left(\boldsymbol{y}_k - \boldsymbol{p}_k\right) \tag{A.12}$$

B. Activation functions

We will in this project, test three different activation functions: the Sigmoid function, the Rectified Linear Unit (ReLU) function and the Leaky ReLU function.
The Sigmoid function, and its first derivative (eq. 8.22 in [3], is given as

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \tag{B.1}$$

$$\sigma'(z) = \frac{e^{-z}}{\left(1 + e^{-z}\right)^2} = \sigma(z)\left[1 - \sigma(z)\right]. \tag{B.2}$$

The ReLU function, and its first derivative (eq. 8.23 in [3], is given as

$$\mathrm{ReLU}(z) = \max(0, z), \tag{B.3}$$

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}. \tag{B.4}$$

The Leaky ReLU function, and its first derivative (eq. 8.24 in [3], is given as

$$\text{LReLU}(z) = \begin{cases} z, & z > 0 \\ \alpha z, & z \leq 0 \end{cases}, \quad \alpha = 0.01 \tag{B.5}$$

$$\text{LReLU}'(z) = \begin{cases} 1, & z > 0 \\ \alpha, & z < 0 \end{cases}. \tag{B.6}$$

III. METHODS

A. Data

In this project we test our neural network on a regression problem, using the Runge function, and a classification problem, using the MNIST database.

As in project 1 [2], we generate $n = 100$ points uniformly in $x \in [-1, 1]$ and targets $y = f(x) + \epsilon$, where f is the Runge function and $\sigma \sim \mathcal{N}(0, \sigma^2)$, with $\sigma^2 = 0.01$ is the noise. The data is split into 80 % training and 20 % test data, with the same split as project 1, so that the network and OLS are evaluated on the same points. The network takes in the raw x data as its only input feature, as it learns its own features. The input is scaled and centred, using the mean and standard deviation of the training data only.

For the classification problem we use the MNIST data set. We use the standard split into 60 000 training and 10 000 test images. Each 28×28 image is flattened to a vector of 784 features, and the pixel values are scaled to $[0, 1]$. The labels are one-hot encoded for the multiclass cross entropy. The test images is used only once for the final evaluation.

B. Implementation of the network

The network is implemented as a class `NeuralNetwork`, which stores the weights and biases of each layer together with one activation function per layer, a cost function and an optional penalty term. Activation function, cost function and penalties are implemented in separate classes, where we can collect the function and its derivative, as written in section II. This allows the network to be set up with any combination of these, without changing the structure of the actual network. The optimizers implemented separately, they take the gradient computed by the network and return the parameter updates.

The samples are stored in the rows of the input matrix, and the weight matrix of layer l has shape (n_{l-1}, n_l), where n_l is the number of nodes in the layer. The feed-forward pass computes

$$\boldsymbol{z}^l = \boldsymbol{a}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l,$$
$$\boldsymbol{a}^l = f(\boldsymbol{z}^l),$$

with $\boldsymbol{a}^0$ being the input (sec. 8.5 in [3]). Each layer therefore has $(n_{l-1} + 1)\, n_l$ parameters.

The weights are drawn from a normal distribution $\mathcal{N}(0, 1/n_{l-1})$, so that the variance of $\boldsymbol{z}^l$ is independent of the number of inputs to the layer. With unscaled weights, $\boldsymbol{z}$ grows with the width, which saturates the sigmoid and makes the gradients vanish. The biases are set to a small constant, 0.01, so that all nodes start in the active region of the ReLU-type activations (sec. 8.9 in [3]).

For regression, the output layer has a single node with a linear activation, $f^L(z) = z$, since the output must be able to take any real value. For classification, the output layers is the same size as the possible classifications with softmax output. In the case MNIST, there are 10 nodes with each node representing one digit.

The gradients are computed with backpropagation, and the error of the output layer is

$$\boldsymbol{\delta}^L = f'(\boldsymbol{z}^L) \circ \frac{\partial C}{\partial \boldsymbol{a}^L},$$

and it is propagated backward through the layers as

$$\boldsymbol{\delta}^l = \left(\boldsymbol{W}^{l+1}\boldsymbol{\delta}^{l+1}\right) \circ f'(\boldsymbol{z}^l), \tag{B.1}$$

giving the gradients

$$\frac{\partial C}{\partial w_{ij}^l} = \delta_j^l a_i^{l-1},$$
$$\frac{\partial C}{\partial b_j^l} = \delta_j^l.$$

Unrolling (Equation B.1) down to layer $\boldsymbol{l}$ gives,

$$\boldsymbol{\delta}^l = \left[\prod_{k=l}^{L-1} \text{diag}\left(f'(\boldsymbol{z}^k)\right) \boldsymbol{W}^{k+1}\right] \boldsymbol{\delta}^L, \tag{B.2}$$

a product of $\boldsymbol{L} - \boldsymbol{l}$ factors. If these factors are lower than one, they usually shrink geometrically, if they're above one they grow geometrically. Neither is survivable [5].

The derivative of the softmax function is not elementwise, so for the softmax output combined with the cross entropy we instead use the simplified output error $\boldsymbol{\delta}^L = \boldsymbol{P} - \boldsymbol{Y}$ (sec. 8.8.4 in [3]).

When an ℓ_1 or ℓ_2 penalty is used, its gradient Equation A.6 or Equation A.4 is added to to the gradient of each weight matrix. The ℓ_2 penalty shrinks the weights towards zero, whereas the ℓ_1 penalty can set a small weights to zero. The penalty is applied to the weights only and not to the biases, since the biases only shift the activations and do not contribute to the flexibility of the model.

1. *Verification*

To verify the backpropagation, we compare the gradients it gives with the gradients of the same cost function

computed by automatic differentiation with `jax.grad` [6]. The comparison is made for the initial weights, and we report the largest absolute difference over all weights and biases (Section IV A).

C. Training and optimizers

In project 1, the cost function could be found analytically and gave the linear regressor a starting point due to the cost function being convex as a result of the relatively simple gradient landscape. For a neural network this is no longer valid as the non-linearity of the activation functions in hidden layers make the cost function non-convex, with no closed form solution. The result depends entirely on how the optimizer descends through the cost function. The initial weights, the learning rate and the size of the minibatches decide both whether the training converges at all, and to which minimum. Incorrectly chosen learning rates leads to divergence or convergence in flat regions. Different initial weights can lead to different final solutions. For that reason, same initial weights are used across methods and learning rates, and is why we repeat key results for several seeds.

The network is trained with the gradient methods from project 1, adapted to update the weights and biases of every layer. We will use: plain gradient descent (GD), stochastic gradient descent (SGD), and SGD with adaptive learning rates of RMSprop and Adam (all these methods and their equations are described in project 1 [2]). GD computes the gradient from the full training set, 80 points, and makes one update per epoch, while the stochastic methods use minibatches of 16 samples, giving $80/16 = 5$ updates per epoch for the Runge data. For MNIST, the updates per epoch are set as in table VIII. For RMSprop we use the decay rate of $\rho = 0.9$, and for Adam the decay rates $\beta_1 = 0.9$ and $\beta_2 = 0.999$, with $\epsilon = 10^{-8}$ in both to avoid division by zero. These values are the typical values, given in [3], and only the learning rate γ is tuned. Unless stated otherwise, the networks are trained for 500 epochs.

In each epoch, the training data is shuffled and divided into minibatches. For each minibatch, the gradients are computed with backpropagation, the optimizer computes the update, and the weights and biases are updated.

We used a fixed seed (2026) for data generation, the train-test split, weight initialisation and minibatch shuffling. The same random numbers are drawn on every run and the results are reproducible. Differences in results come from the hyperparameters or optimizers, not from randomness. However, results can still depend on the initial weights and minibatch order, so we averaged over five seeds when comparing architectures. We only did this when comparing architectures since it's computationally costly as an extra seed repeats a full run.

D. Experiments and hyperparameter search

To find a suitable learning rate for each gradient method, we perform a grid search over nine logarithmically spaced values of the learning rate $\gamma \in \{10^{-4}, 10^{0}\}$. For each combination of learning rate and gradient method (GD, SGD, SGD + RMSprop, SGD + Adam), we train a network with one hidden layer of 50 sigmoid nodes for 500 epochs, starting from the same initial weights, and record the test MSE after the last epoch. Runs where the MSE grows without bounds are marked as diverged.

For each method, the learning rate with the lowest MSE is chosen, and the best overall is used in further analysis. Note that the learning rate is selected on the test MSE, rather than on a separate validation set, so the reported test MSE of the selected model is slightly optimistic.

To save computation time we reused the best optimizer for sigmoid (SGD + Adam) for ReLU and LeakyReLU and swept the same over learning rates. Similarly to sigmoid, γ was chosen from the lowest test MSE, so this MSE is slightly optimistic.

With γ fixed, we varied the network size with 1-5 hidden layers and 1-100 nodes per layer for each of the activation function without a validation set for a fair comparison with Project 1. Overfitting appears as a growing gap between training and test error [7], so we graphed both over long training runs. We also trained deeper networks to test for unstable gradients.

With the chosen architecture, we tuned λ and γ for the ℓ_1 and ℓ_2 penalties using 5-fold cross-validation on the training data, scaling inside each fold to avoid leakage and withholding a test set until the very end. We pick the most regularized combination (largest λ, smallest γ) whose MSE is within one standard error (1SE) of the lowest MSE (the 1SE rule), where

$$\mathrm{SE} = \frac{\hat{\sigma}}{\sqrt{k-1}},$$

and $\hat{\sigma}$ is the standard deviation of the validation MSEs over the k = 5 folds derived by Scikit-learn.

The chosen networks are evaluated once on the withheld test set and compared with the Ridge and Lasso using the best parameters from Project 1.

Since cross-validation over 500 epochs is costly, we only use one seed. However, cross-validation gives five different combinations of train and validation splits which will give an estimate of the uncertainty. (see [2] for more on cross-validation).

The training for the MNIST dataset follows a mostly similar approach, but with Multiclass cross-entropy as the cost function instead of MSE. We first run through a coarse grid as in fig. 16 with fewer epochs to get a general idea of suitable parameters for training finer models. In order to minimize time setting up training grids, a wide

grid with 105 combinations (see table VIII for parameters), each with 150 epochs was used. Here we varied all hyperparameters (λ, γ, penalty type, number of nodes per layer) except for the hidden layer activation functions which were all ReLU, with softmax outputs. The models output probability distributions for each digit, where the digit with the highest probability is deemed as the predicted digit during evaluation.

Out of the big grid trained, two models were chosen. The best model, meaning the model with the highest accuracy, and the best cheapest model, being that within the 1SE rule. We could group our models into four 'clusters' based on fitting time and accuracy, where we chose our cheapest cluster model (marked in fig. 9b). Parameters for the best and the cheapest cluster model were extracted and retrained using the Keras module in TensorFlow (TF). From hereof three sets of models will be discussed. 1. the rough grid models, 2. *Our Own Best Model*, **OOBM**, and the best model of the significantly cheaper model referred to as *Our Own Cheapest Model*, **OOCM**, and 3. the TF models based OOBM and OOCM respectively named **TFBM** and **TFCM**.

E. Evaluation and baselines

The performance of the regression models is measured by the mean squared error (MSE, Equation A.1) on the test set, computed on the scaled targets. Since centring the targets only shift them by a constant, the MSE is the same as on the original scale. The data contains noise with variance $\sigma^2 = 0.01$. It is the noise floor, the lowest MSE any model can we expected to reach, and compare the models by how close they come to it.

As a baseline for the regression, we use OLS with the degree that gave the lowest MSE in project 1 (degree 8). The OLS solution is computed in closed form with the same train/test split and the same scaling as for the network. The two methods are evaluated on exactly the same test points.

When we use the ℓ_1 and ℓ_2 norms, the baselines are Lasso and Ridge regression from Project 1, with degree = 11 and $\lambda = 7.28 \times 10^{-5}$ and $\lambda = 3.56 \times 10^{-5}$ respectively.

For MNIST we set a logistic regression model, **LG**, as the baseline. It is the bare minimum of a neural network. We are looking for any increase in the reported metrics from the baseline, which is what really determines how good our model is.

Evaluating classifiers is slightly more complicated. Though a low cost is ideal, a classifier is better evaluated by it confusion matrix (CM), where number of combinations of true labels vs predictions are plotted. A perfect result would be entirely diagonal from top left to bottom right as it represents true positives(TP). All other labels represent false positives(FP). We can also have true negatives(TN) and false negatives(FN). What is defined as a correct prediction can vary as threshold can be manually defined. To investigate the threshold, we use the receiver operating characteristic (ROC), where the threshold for a correct prediction is varied. The Area Under the Receiver Operating Characteristic Curve (ROC AUC) tells us how well a binary classifier does, where a value of 1 is perfect and 0.5 means random guessing[8].

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{E.1}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{E.2}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{E.3}$$

$$\begin{aligned} F_1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \\ &= \frac{2TP}{2TP + FP + FN} \end{aligned} \tag{E.4}$$

Accuracy (eq. (E.1)) is a decent measure when the classes are balanced, as in our case. It reports all correct predictions over the total. Precision (eq. (E.2)) reports fraction of TPs of all positives, while recall (eq. (E.3)) true positives were found out of all positives, including the ones wrongly labelled as negative. Precision and recall often go hand in hand, where if the threshold for a positive is lowered the precision increases, but recall decreases as false negatives increase too. Which is most important depends on the goal of the model. F1 (eq. (E.4)) combines the two, describing the models ability to correctly identify positives, and is only high when both precision and recall are. One has to interpret accuracy, precision, recall and the F1-score in light of the CM[3].

IV. RESULTS AND DISCUSSION

A. Verification of the implementation

The gradient given by backpropagation is checked against the automatic differentiation of `jax.grad` from the JAX library [6], with both gradients given by the initial weights of each network, on the scaled training data. For a Runge network with one hidden layer of 50 sigmoid nodes, the largest difference is $2.8 \cdot 10^{-16}$ Table II shows gradient checks across four differently sized networks all staying between 10^{-17} and 10^{-15}.

These differences are of the same order as the machine precision of 64-bit floating point numbers, which is about $2.2 \cdot 10^{-16}$. This tells us that the gradients we get from backpropagation are correct, and the results will therefore not be affected by errors in the gradient of our network.

B. Regression with a sigmoid network

1. Learning rate sweep and gradient method

When using sigmoid-only network, to get the most accurate result, we want to use the optimizer method that performs the best, along with the best learning rate. To find an optimal value we do a learning rate sweep in the interval $\gamma \in [10^{-4}, 10^{0}]$. Figure 1 shows the test MSE of the 1×50 sigmoid network for each combination of learning rate and gradient method, and Table I gives the best learning rate for each method. The darker colour shows the lower MSE, and the methods with adaptive learning rates clearly perform better. SGD + Adam reaches the lowest test MSE, 0.0127 at $\gamma = 3.2 \cdot 10^{-3}$.

Part of the difference between GD and SGD comes from the number of updates, with 80 training points and minibatches of 16, SGD makes five updates per epoch, while GD makes one. In the following we will use SGD + Adam with $\gamma = 3.2 \cdot 10^{-3}$.

Table I: Best learning rate γ and the corresponding test MSE for each gradient method, from a scan over $\gamma \in [10^{-4}, 10^{0}]$. All runs use a 1×50 sigmoid network.

Optimizer	Best γ	Test MSE
GD (full batch)	$1.0 \cdot 10^{-1}$	0.0432
SGD	$1.0 \cdot 10^{-1}$	0.0229
SGD + RMSprop	$3.2 \cdot 10^{-2}$	0.0161
SGD + Adam	$3.2 \cdot 10^{-3}$	**0.0127**

2. Network size

Table II shows that adding a second hidden or increasing the number of nodes increases the number of parameters, but leaves the MSE unchanged.

In Figure 2b the networks with two hidden layers performs better at the peak of the function, and in Figure 3b they reach their lowest MSE in fewer epochs. With a single seed, and only 100 data points, the differences between the different sized networks are smaller than we can resolve. For this smooth, one-dimensional function, one hidden layer with 50 nodes is therefore sufficient, when using sigmoid.

3. Comparison with OLS

Table III compares the best network with OLS using the best polynomial degree from project 1. The network reaches a test MSE of 0.0131, compared with 0.0187 for OLS. The noise floor $\sigma^2 = 0.01$ is also included, the network comes within 0.0031, and OLS within 0.0087 of the noise floor. The network reduces the error by about two thirds of OLS, even before the learning rate is tuned.

Table II: Train and test MSE for different network sizes (hidden layers $\times$ nodes per layer), trained with Adam ($\gamma = 3.2 \cdot 10^{-3}$). The last column is the largest difference between the gradients from our backpropagation and from `jax.grad`.

Network	Parameters	Train MSE	Test MSE	Gradient check
1×50	151	0.0147	0.0127	$2.8 \cdot 10^{-16}$
1×100	301	0.0148	0.0127	$4.4 \cdot 10^{-16}$
2×50	2 701	0.0140	0.0129	$2.2 \cdot 10^{-16}$
2×100	10 401	0.0138	0.0127	$2.8 \cdot 10^{-17}$

The number of parameters for the different methods show that the neural network is more flexible than OLS, but this comes at a cost. OLS has 9 parameters and a closed form solution, while the network has 151 parameters, and depends on the random initialization.

In Figure 2a the difference between the fit to the Runge function, for the neural network and OLS, becomes more clear. OLS underestimates the peak of the function, and oscillates near $x = \pm 1$, whereas the network follows the true function over the whole interval. The MSE as a function of epoch for the training and test data for this neural network is found in Figure 3a, where we can see that it drops the first 100 epochs, and then stabilizes above the noise floor.

Table III: Test MSE for the neural network and for OLS with the best polynomial degree from project 1, on the same train/test split of the Runge data ($n = 100$, $\sigma = 0.1$). Network is trained with Adam, with the learning rate $\gamma = 0.01$. The noise floor σ^2 is the lowest MSE a perfect model can reach.

Model	Parameters	Test MSE
OLS, degree 8	9	0.0187
Neural network, 1×50	151	0.0131
Noise floor σ^2	–	0.0100

C. Activation functions and depth

1. Learning rate sweep for ReLU and LeakyReLU

With SGD + Adam the best learning rate for ReLU and LeakyReLU is $\gamma = 0.01$, with test MSEs of 0.0157 and 0.0158. Sigmoid does best at $\gamma = 3.2 \times 10^{-3}$ and reaches a lower test MSE of 0.0128. The ReLU family tolerates larger learning rates because its derivative is 1 for all positive inputs (Equation B.4, Equation B.6). The sigmoid derivative (Equation B.2) never exceeds 1/4 and goes to zero for large $|z|$, so large updates push sigmoid nodes into saturation, where they learn very slowly [5]. However, a higher tolerated γ does not mean a better fit.

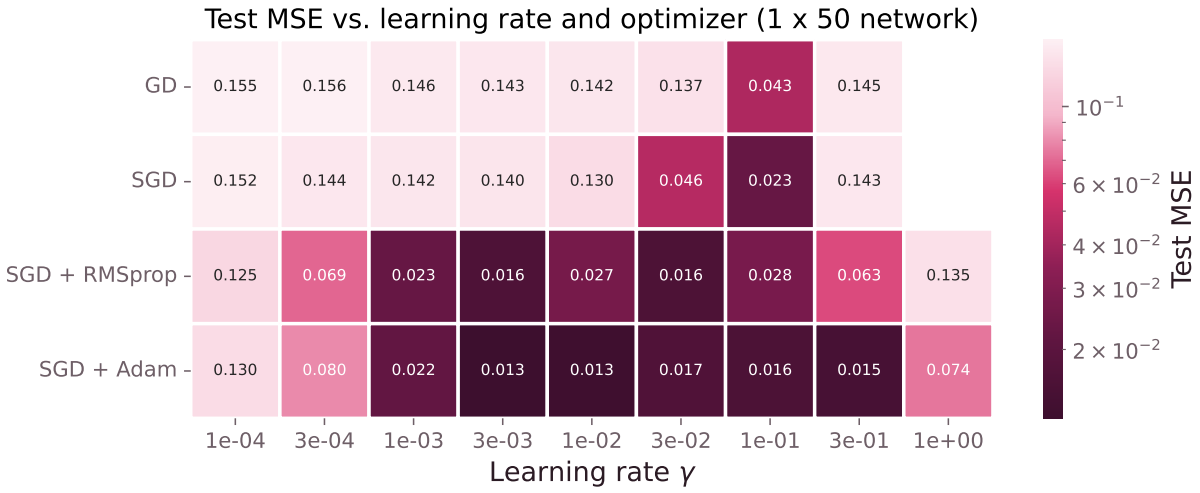


Figure 1: Test MSE on the Runge data after 500 epochs over a grid of learning rates and gradient methods for the 1×50 sigmoid network with a linear output show that the adaptive methods reach the lowest MSE and work well over a wider range of γ, while plain GD and SGD need larger learning rates and diverge for $\gamma = 1$.

2. *Testing different network sizes*

Figure 4 show that networks with one node per layer can be discarded for all activation functions. In the case of ReLU with 1 hidden layer, the MSE remains large and is a clear case of dying ReLU.Beyond these small networks, we see that depth and width have little effect on the test MSE. The best test MSE lies between 0.010 and 0.014 for all activation functions. The best networks per activation function over 5 seeds were Sigmoid 2×25, ReLU 5×50 and LeakyReLU 5×100, all within 1SE of each other. Computation time grows with both depth and width since both increase the number of parameters (Figure 15).

In deeper networks, the test MSE for sigmoid rises with depth, whereas ReLU and LeakyReLU stay stable (Figure 6). For sigmoid, the gradients reaching the early layers leave the weights essentially unchanged. The saturated nodes contribute almost nothing and the networks become almost untrainable. The derivative for ReLU and LeakyReLU is 1 for positive inputs, so the factor in (Equation B.2) neither shrink or explode the gradient.

3. *Testing for overfitting*

Figure 6 show that every activation function eventually overfits. Sigmoid overfits at 2700 epochs for the $((4 \times 100)$ and ReLU (4×25) and LeakyReLU (4×100) after about 3000-4000 epochs. For our main analysis, we've used 500 epochs. Figure 13, show that SGD + Adam has converged by then. And these models show that 500 epochs is well before any overfitting.

4. *Choosing a fitting architecture*

Since adding layers and nodes apart from the first barely changes the test MSE Figure 4, we choose a 2×50 network. It fits the Runge functions as well as larger networks, is cheaper to train due to less parameters and it is far from the depths of 8 or more where vanishing gradients appear.

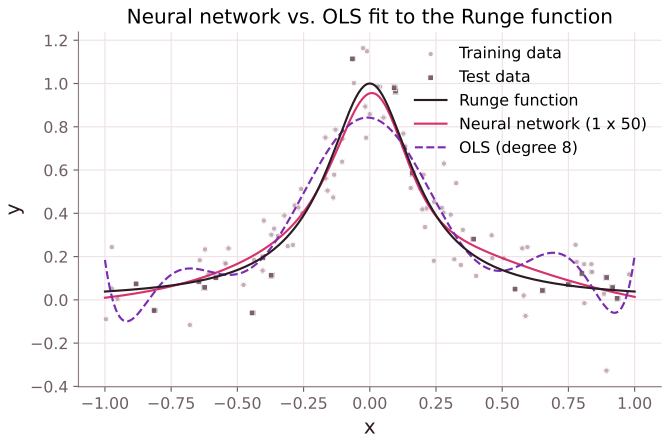
We continue with sigmoid and LeakyReLU since sigmoid is a fundamentally different activation function compared to ReLU and LeakyReLU, which are in the same family. As LeakyReLU avoids the dying ReLU problem which seen from Figure 4 from the smallest networks.

D. ℓ_1 and ℓ_2 norms

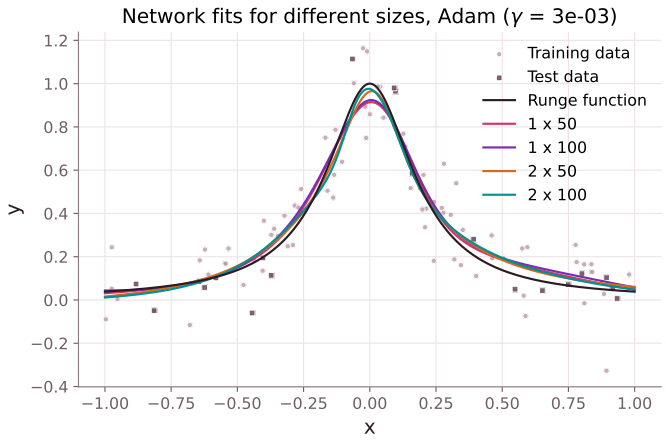
Figure 7 we see that sigmoid is more sensitive to higher λ's compared to LeakyReLU, with much higher MSEs at $\lambda = 0.001$ and $\lambda = 0.01$ for both ℓ_2 and ℓ_1 norms. With Table IV in mind, the lowest 5-fold CV MSE is similar for both activation functions.. From Figure 8, the majority of the validation MSEs derived from lower penalties λ follow the 1SE rule for lowest CV MSE.

Using the hyperparameters from the 1SE rule, we get the test MSEs as 0.0163 and 0.0147 for ℓ_2 and ℓ_1 for sigmoid and 0.0151 and 0.0156 for ℓ_2 and ℓ_1 for LeakyReLU. This is a large improvement from the test MSEs from Project 1 using the best degree and best λ which gave 0.0219 and 0.0220 for Ridge and Lasso, respectively.

From Figure 8 we see that the neural networks using the ℓ_2 and ℓ_1 norms will visibly flatten the oscillations at the edges of the Runge function that Ridge and Lasso could not from Project 1. We see that the sigmoid re-

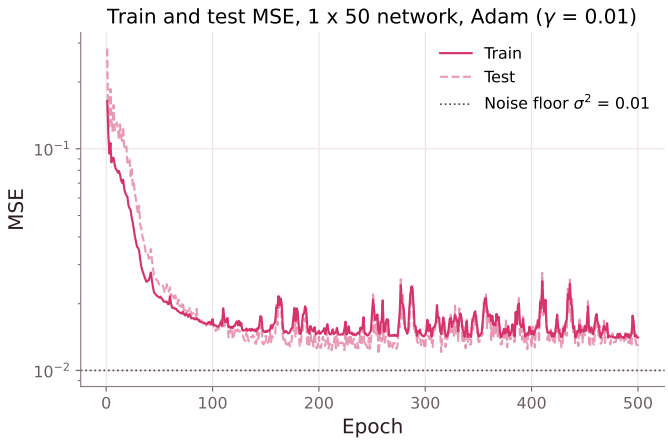


(a) Neural network and OLS.

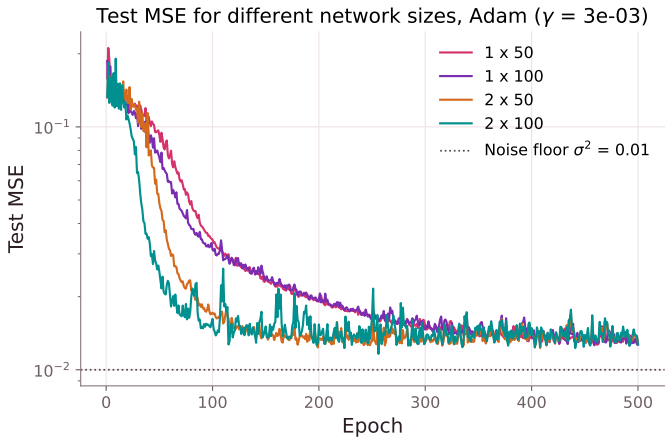


(b) Different network sizes.

Figure 2: (a) A 1×50 sigmoid network trained with Adam ($\gamma = 0.01$), compared with OLS . The polynomial oscillates near $x = \pm 1$ and underestimates the peak, while the network does not. (b) Networks with one or two hidden layers of 50 or 100 sigmoid nodes, all trained with Adam ($\gamma = 3.2 \cdot 10^{-3}$). The networks with more layers perform better at the peak.



(a) Train and test MSE, 1×50 network.



(b) Test MSE for different network sizes.

Figure 3: The noise floor at $\sigma^2 = 0.01$ is the lowest MSE a perfect model can reach. (a) The 1×50 sigmoid network from Fig. 2a, trained with Adam ($\gamma = 0.01$). (b) Test MSE for the four network sizes from Fig. 2b, trained with Adam ($\gamma = 3.2 \cdot 10^{-3}$). All sizes converge to a test MSE of about 0.013.

Table IV: The lowest cross-validated MSE is nearly the same for both activation functions in the 2 x 50 network for both penalties. Whereas the one standard error choices for sigmoid is higher than the one standard error choices for LeakyReLU

		Lowest			1SE		
Activation	Norm	λ	γ	MSE (10^{-2})	λ	γ	MSE (10^{-2})
Sigmoid	L2	0	10^{-3}	1.77 ± 0.38	10^{-4}	10^{-3}	2.00 ± 0.35
	L1	10^{-4}	10^{-1}	1.74 ± 0.36	10^{-4}	10^{-3}	1.89 ± 0.36
LeakyReLU	L2	10^{-3}	10^{-3}	1.73 ± 0.39	10^{-3}	10^{-3}	1.73 ± 0.39
	L1	10^{-4}	10^{-3}	1.73 ± 0.37	10^{-3}	10^{-3}	1.81 ± 0.38

Table V: The neural network derive better test MSE from both activation functions using the ℓ_1 and ℓ_2 norms using the one standard error choice compared to the Lasso and Ridge from Project 1 with the most optimal parameters

Model	λ	γ	CV MSE (10^{-2})	Test MSE (10^{-2})
NN Sigmoid L2	10^{-4}	10^{-3}	2.00 ± 0.35	1.63
NN Sigmoid L1	10^{-4}	10^{-3}	1.89 ± 0.36	**1.47**
NN LeakyReLU L2	10^{-3}	10^{-3}	1.73 ± 0.39	1.51
NN LeakyReLU L1	10^{-3}	10^{-3}	1.81 ± 0.38	1.56
Ridge (deg. 11)	$3.6 \cdot 10^{-5}$	–	–	2.19
Lasso (deg. 11)	$7.3 \cdot 10^{-5}$	–	–	2.20

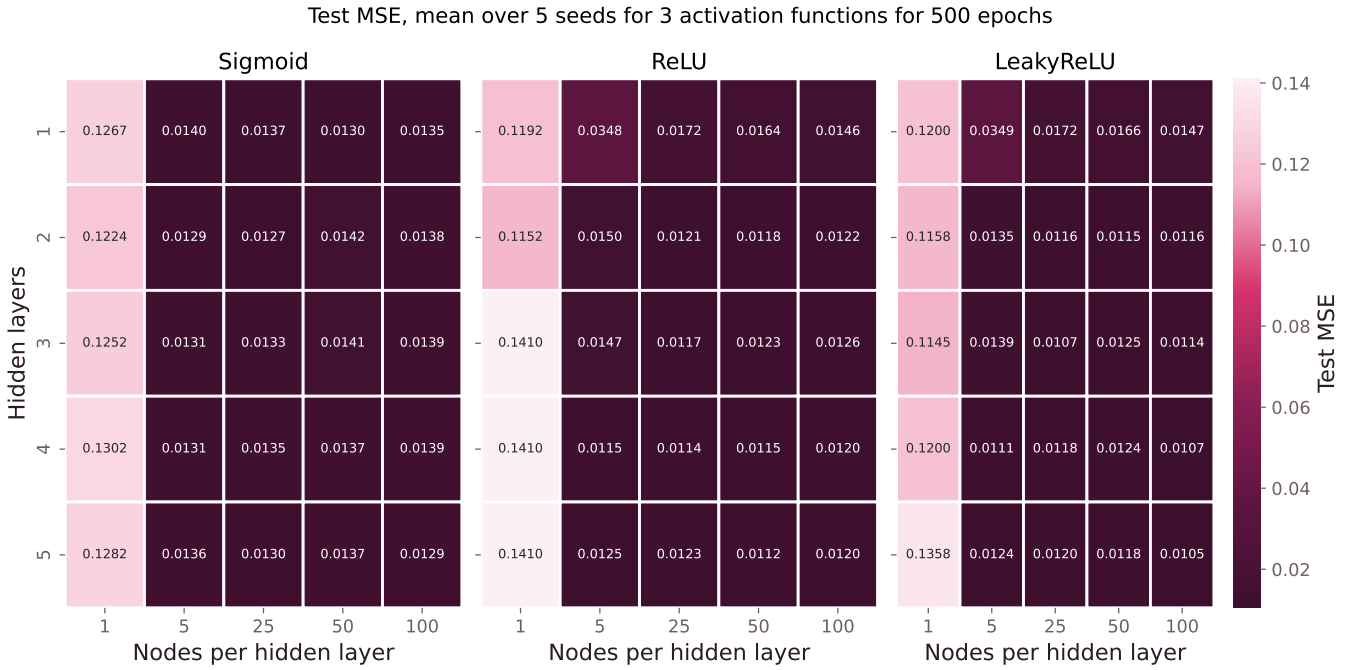


Figure 4: Excluding the small networks of 1-5 nodes and 1 layer, the number of nodes and the depth of the network does not significantly improve the test MSE for the Runge data. For networks on this size, the activation functions are nearly indistinguishable.

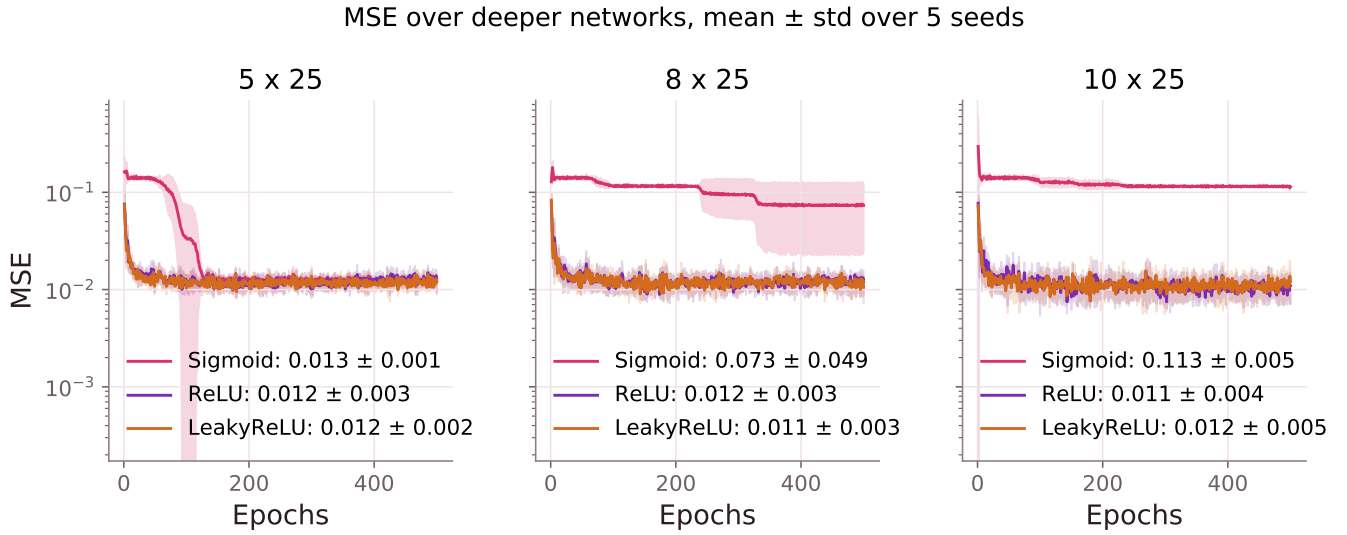
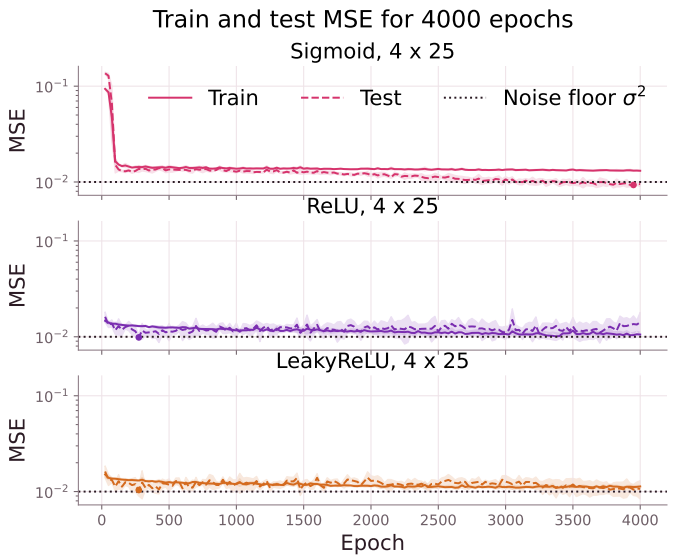
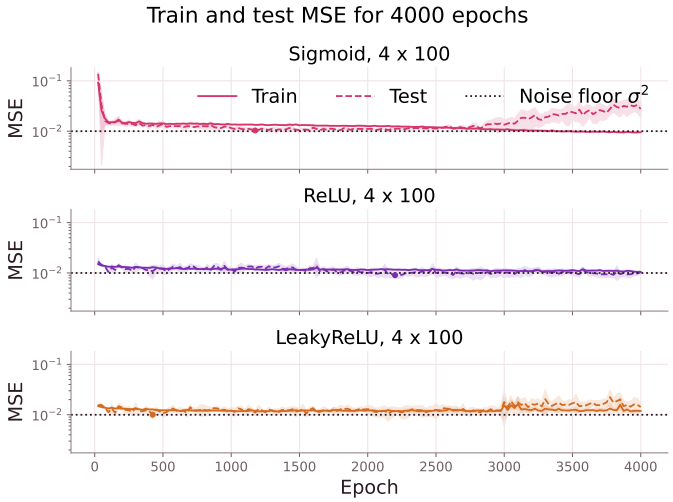


Figure 5: The depth of the network increases the test MSE of the sigmoid activation function due to vanishing gradients, wheras ReLU and LeakyReLU stay unchanged



(a) Long training on 4 x 25 network



(b) Long training on 4 x 100 network

Figure 6: Sigmoid and LeakyReLU overfits for networks with more nodes whereas ReLU seemingly overfits for a network with less nodes. Both have same hidden layers. None of the overfitting happens close to 500 epochs which is the hyperparameter we've used for our main analysis.

draws a smoother functions whereas LeakyReLU is more jagged but still follows the same shape.

E. Classification of MNIST

The baseline logistic regression model (LG), performs badly and frequently gives FPs (fig. 9a), with an average accuracy of 0.93 (table VI), but is significantly better than a random prediction. Its recall is especially bad, with key examples being digit 2, 5 and 8 with recall at 0.905 , 0.871 and 0.894.

Models OOCM, OOBM, TFCM and TFBM aim to improve upon these rates, with CMs and ROCs in (fig. 10 showing that they do. Their classification scores in table VII visualized in fig. 11 further amplify this result, particularly for the cases of digits 2, 5 and 3 which did particularly bad for LG. The accuracy OOCM, OOBM, TFCM and TFBM is 0.976, 0.979, 0.981 and 0.981 respectively. When comparing parameters of the models in different clusters we see that more hidden layers and more nodes generally gave a higher accuracy, but were still highly dependent on other parameters. No penalty type, penalty or learning rate alone determined the performance of the model.

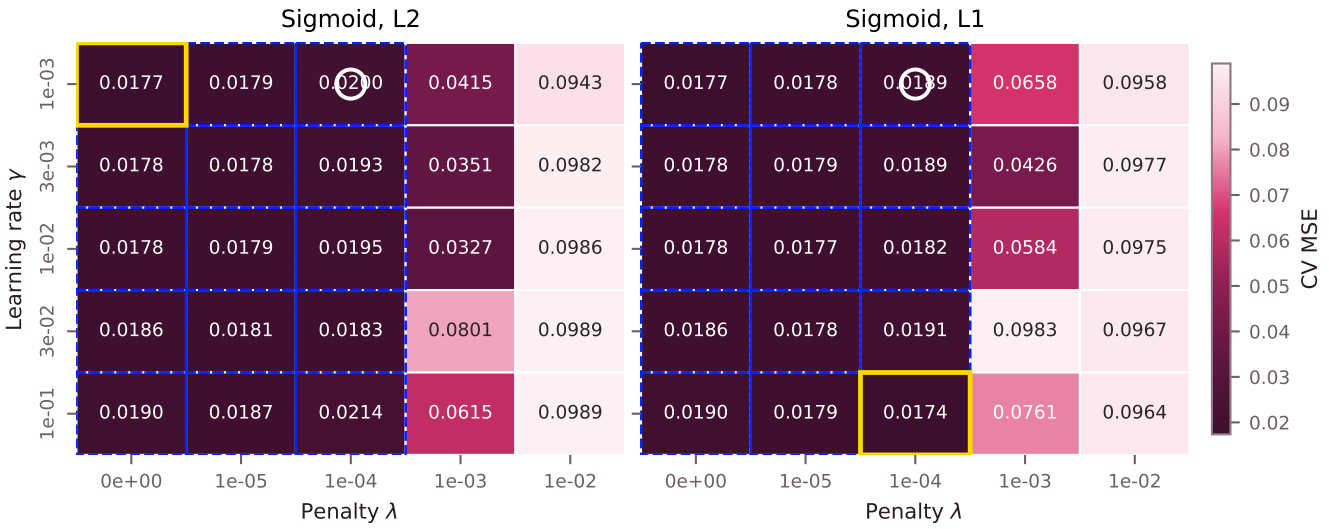
When comparing fit time with accuracy (fig. 9b) OOBM is 2x as computationally expensive as OOCM, despite gain in accuracy being only 0.003. TFCM and TFBM which were trained using TensorFlow both gave higher accuracy despite parameters being the same as fr OOCM and OOBM. This tells us that our neural networks have numerical quirks that make the TF models and our own models differ. If one wanted the best of the best models, one would not train a TF model based on another network. One would instead do their own set of numerical experiments to determine the best parameters for a TF model.

With fig. 11 in mind, one can deem TFBM to be the best, as its F1 scores remain mostly the highest or second highest and accuracy is at 0.981. Choosing TFBM cannot be strongly argued for though, as the total increase is relatively small. Computational cost was not logged, though TFCM was quicker to train and fit than TFBM. Digits wrongly classified by TFBM seen in fig. 18 are at times hard to determine for human eyes too.

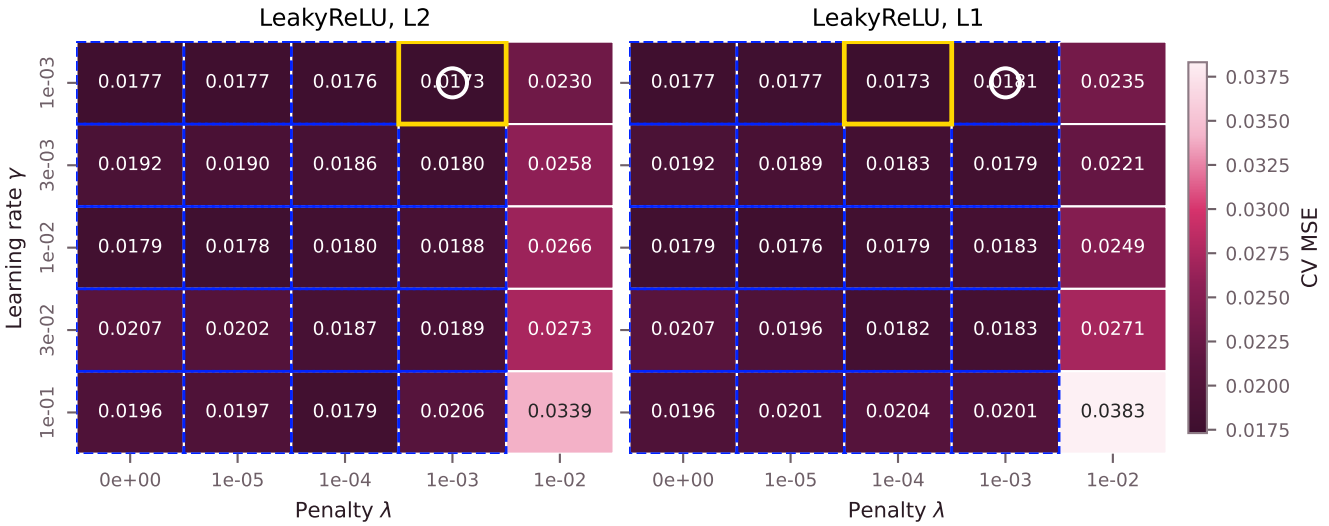
Time in itself is not a good metric to gauge computational cost, as available computational power differs per device. All MNIST models were trained in the same environment with the same computational resources and can therefore be compared. In real-world applications fitting time can be the limiter, deeming OOCM as the most applicable model.

When training a model of higher complexity than needed for MNIST, it is unreasonable to do it with such a wide grid as done here. One would ideally run more architecture, penalty and epoch numerical experiments as done for Runge. Due to time constraints, we preferred to throw a wide net instead, though it cost us computational resources instead. This approach will likely be tweaked significantly for future work.

The effect of our choice to not test further can be seen in the training history in fig. 17. The best hyperparameters were likely not found, as the cost for OOBM stagnates at 10^{-1}, when for other models it surpasses 10^{-2}. This may also have to do with the choice of activation function, as for all hidden layers ReLU was chosen, which is known to occasionally stagnate. A better result could have been achieved with LeakyReLU, or a combination



(a) Sigmoid



(b) LeakyReLU

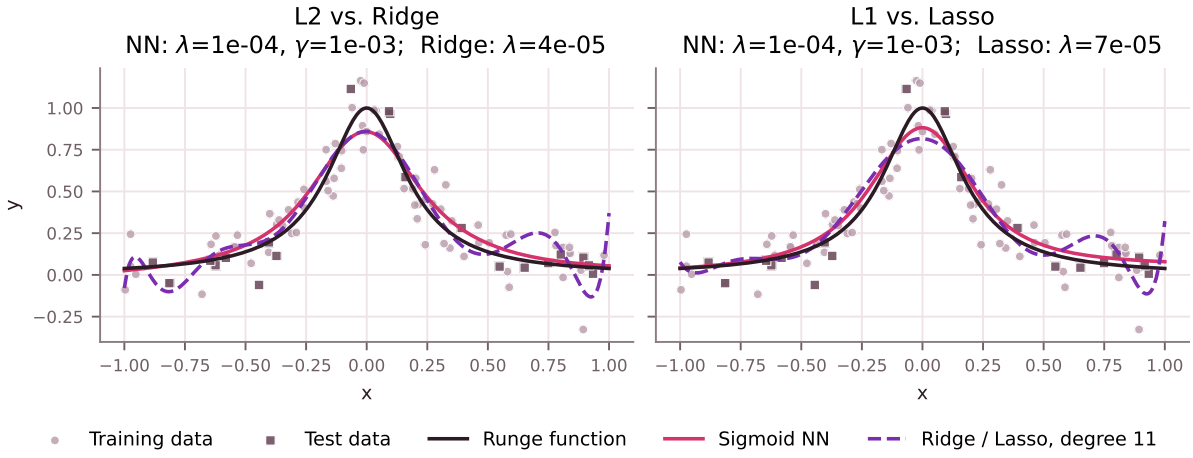
Figure 7: 5-fold CV MSE for the 2x50 network with L2 and L1 penalty. The lowest MSE is marked in gold and is nearly the same for both activation functions. All MSEs within one standard error of the lowest are marked with blue dashed borders. The MSE marked with a white circle is the MSE chosen via the 1SE rule. The 1SE rule gave higher MSEs for Sigmoid than for LeakyReLU.

of several activation functions for the hidden layers.

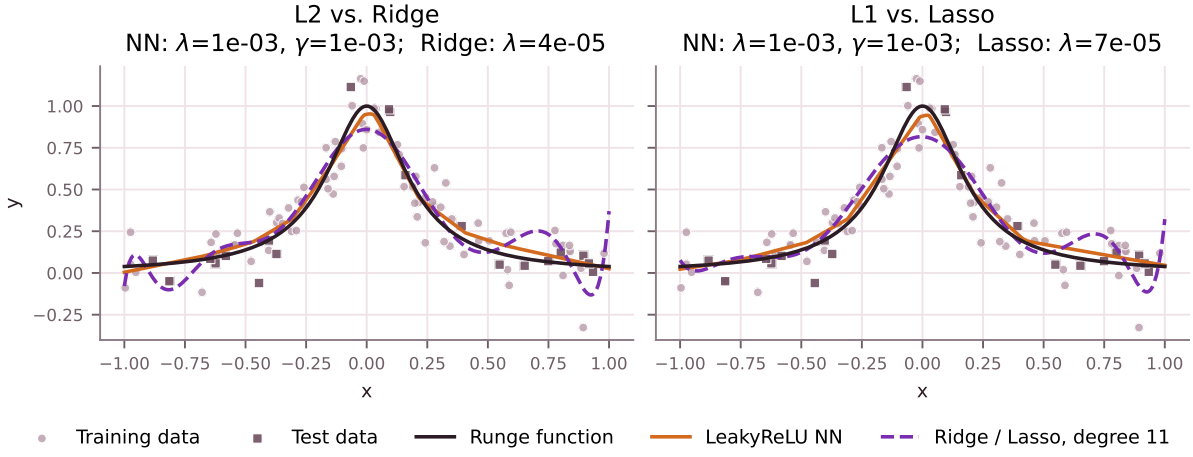
V. CONCLUSION

SGD + Adam gave the lowest test MSE among the gradient methods and was used for all regression runs. Apart from very small networks (1-5 nodes), network size had little effect on the test MSE and for 1-5 hidden layers the thee activation functions were nearly indistinguishable. Sigmoid worsened due to vanishing gradients, whereas ReLU and LeakyReLU stayed stable. We chose a 2×50 network it is within 1 std of the lowest test MSE and cheaper to train. Over-fitting only set in after 2700-4000 epochs, which is beyond the 500 epochs used for main analysis.

With ℓ_1 and ℓ_2 penalties, the networks reached test MSEs of 0.0147-0.0163 compared with 0.0219 for Lasso and 0.0219 for Ridge from Project 1. The networks visibly dampened the oscillations near the endpoints of the Runge function, which regularization in Project 1 could not. Sigmoid gave the best fit as its smooth output matches the smooth Runge function.



(a) Sigmoid



(b) LeakyReLU

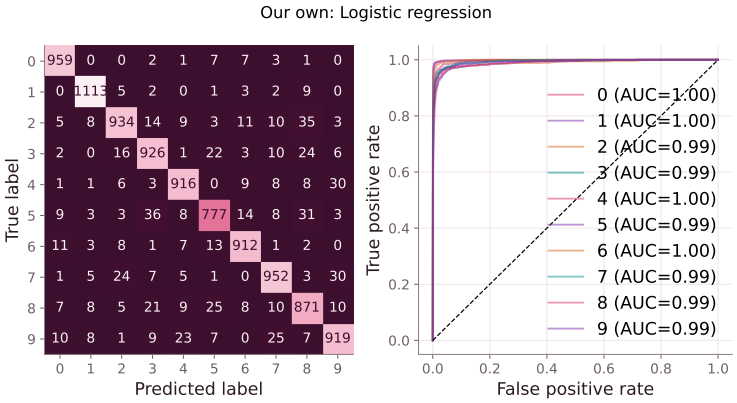
Figure 8: The one standard error networks with ℓ_2 and ℓ_1 penalty compared with Ridge and Lasso using their optimal parameters from Project 1. Both activation functions perform better than Ridge and Lasso on the Runge data. The networks visibly dampen the oscillations near the endpoints, which was the main problem when fitting the Runge function in Project 1.

However these results have come at a cost. The network has many hyperparameters (epochs, optimizer, learning rate, depth, nodes, activation function and λ). It is much slower to train and tune compared to Ridge and Lasso.

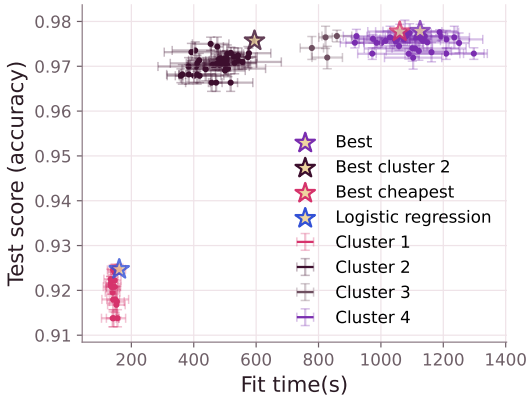
The results are mainly limited in three ways. By the optimizer, learning rate and architectures were chosen by the lowest test MSE, without a validation set, so the MSEs are slightly optimistic. By few data points as we kept the 100 data points to Project 1 for direct comparison, which only gave 20 test points. And by ℓ_1 and ℓ_2 fits using only one seed to save computation time. In future work we would use more data points, split into training, validation and test, where test is used only once at the end and average the ℓ_1 and ℓ_2 results over five seeds as we did for the architectures.

In the case of MNIST, similar trends were observed, but amount of hidden layers and nodes were the main indicator of computational cost. Otherwise interplay between hyperparameters gave different accuracies and fitting times. It was found that our best model has an accuracy of 97.9%, while the best TensorFlow model was at 98.1%, despite being trained on the same hyperparams. A numerical difference in how the models are trained is likely at fault. A better model could be developed using TensorFlow yielding higher accuracy and lower computational cost.

A common takeaway for both the Runge and MNIST neural networks is that accuracy costs computational power. A network with many parameters, layers and nodes is much slower to train and fit. Real-world applications may not afford the time for complex models, making us chose the less accurate models as a trade-off.

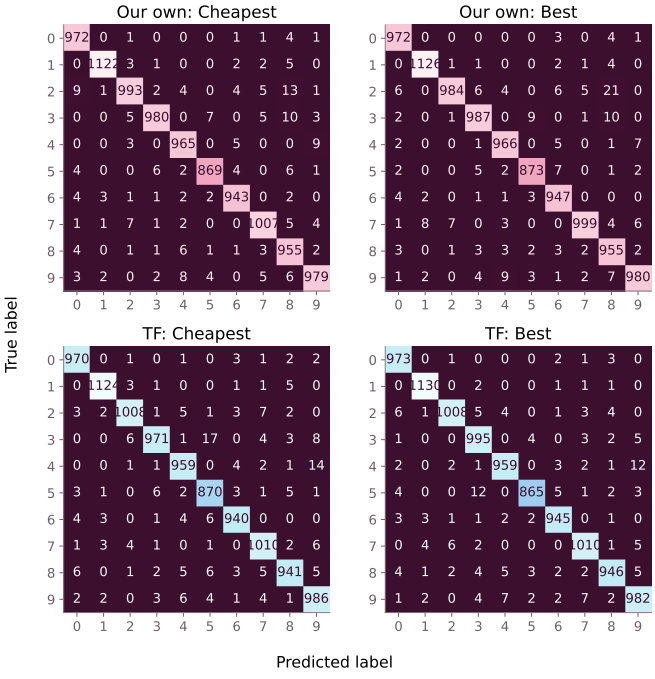


(a) Our logistic regression baseline (see table VIII) classifies the worst out of all models. It predicts wrong multiple times on all digits. The ROC AUC is 1 or 0.99 in all cases, meaning it predicts decently well but not perfectly.

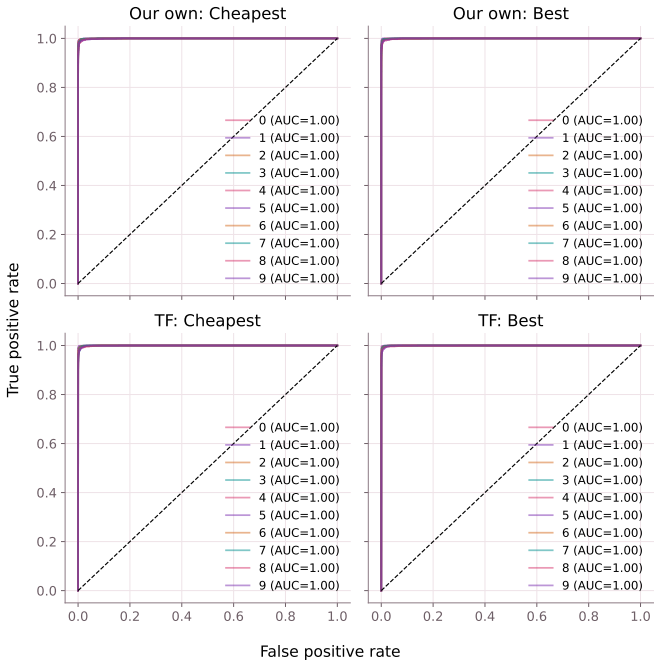


(b) The computational cost vs the score (accuracy) of our own neural network can be grouped into four clusters, with 'Best' denoted as OOBM and 'Best cluster 2' as OOCM. OOCM costs half as much as OOBM. 'Logistic regression' is our baseline.

Figure 9: Baseline logistic regression (a) and fit time versus accuracy for all trained models (b).

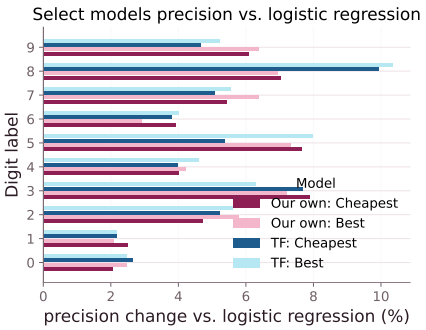


(a) Confusion matrix showing that all four models predict the wrong digit only a fraction of the time, with no clear point of failure with only the matrix as reference. One can point out that OOBM and OOCM fail for similar combinations for predicted and true labels, while TFBM and TFCM do the same but for different combinations of labels.

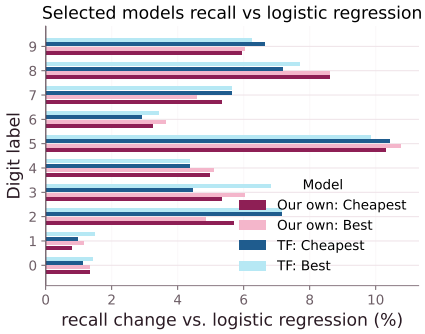


(b) ROCs show that predictions high true positive rates for all thresholds over all models. The results are nearly indistinguishable to the naked eye and all yield an AUC of 1.

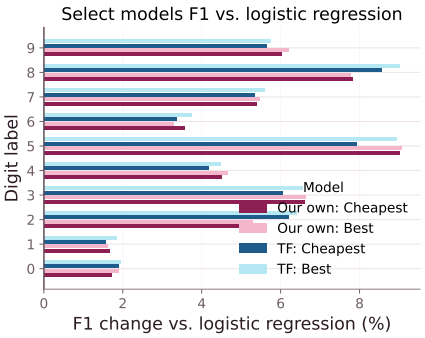
Figure 10: With the CM, it is clear that the models perform differently, but not which performs better. A clear winner cannot be announced.



(a) The TF models outperform for digit 9, but fall somewhat behind on digits 9, 7 and 1 for precision.



(b) The TF and OO models perform differently for each digit, with no clear winner for recall.



(c) F1, which combines recall and precision, shows clear gain for all digits, particularly ones that are visually the closest (8 and 5). All models perform similarly well.

Figure 11: When comparing our four models to the baseline logistic regression they all do better. The gain in recall is perhaps of most significance, as it means less FNs. Further details are in table VII, with the logistic regression model's scores in table VI.

[1] F. Lee, IBM [Online]. Available: https://www.ibm.com/think/topics/neural-networks (2021).
[2] P. X. Amundsen, M. Andersen, and H. Westgaard, *FYS-STK4155: Project 1*, Tech. Rep. (University of Oslo, 2026).
[3] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences* (Springer, 2026).
[4] Y. LeCun, C. Cortes, and C. Burges, ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist **2** (2010).
[5] M. Hjorth-Jesnen, Chapter 8: Neural networks, Github (2026).
[6] JAX library, `https://docs.jax.dev/en/latest/`.
[7] M. Hjorth-Jesnen, Project 2 on machine learning, Github (2026).
[8] SciKit Learn, 3.4. metrics and scoring: quantifying the quality of predictions, `https://scikit-learn.org/stable/modules/model_evaluation.html#roc-metrics` (2026).

ACKNOWLEDGMENTS

We would like to thank our amazing group teacher Amund for amazing help during the amazing group sessions. See Figure 12 for Amund in his element.



Figure 12: Our amazing group teacher Amund Solum Alsvik and some guy standing way too close to him. (Psst, is he bothering you wide cowboy?)

Appendix A: Use of AI/LLM tools

1. Tools used.

Claude (claude.ai, October 2026): Models:

- Opus 5.5, Effort: Medium.
- Copilot GPT-6-Luna

2. Text writing and editing

Section	Level	Notes
Abstract	0	No LLM was used
Introduction	1	Spell checks
Theory	0	No LLM was used
Methods	2	LLM used to find the best way to structure the section. LLM was used to help rewrite more concretely.
Results & Discussion	2	LLM used to format the majority of figures and tables derived from code to LaTeX syntax. LLM used as an academic sparring partner using own ideas and conclusions for some of the discussion. LLM was used to help rewrite more concise.
Conclusion	1	LLM was used to help rewrite more concise.

3. Code generation and assistance

File / notebook	Level	Description
`_Part_a.py`	1	Minor debugging.
`_Part_b.py`	1	Minor debugging.
`_Part_c.ipynb`	2	Minor debugging. LLM used for a majority of the plotting.
`_Part_d.ipynb`	3	Debugging. LLM did all of the plotting and did all of the printing of results when comparing the network to Ridge and Lasso from Project 1.
`_Part_e.py`	3	LLM helped make plotting helper functions, as well as retraining function using keras. Made docstrings for functions.
`_Part_e.ipynb`	3	LLM helped create the fit-time cluster DataFrames and a cluster-colored LaTeX table of model configurations, including mean and standard deviation values. Added plot markers to fit time vs score figure. Helped build a TensorFlow retraining helper based on selected scikit-learn model settings. Added baseline-comparison for F1, recall and precision plot. Helped get wrong predictions out for image plotting and probability dist based on TF code.
`_Part_e_*.py`	3	LLM helped with setting up retraining for our own neural network and TensorFlow networks.
`general_functions.py`	2	LLM rewrote existing code from functions into classes, as this was discovered to be a better way to structure the code after the code was already written. Added helper class for optional passable arguments.
`linear_regression.py`	1	Reused from Project 1 where we used LLM to help debug and comment code.
`neural_networks.py`	2	LLM rewrote existing code from functions into classes, as this was discovered to be a better way to structure the code after the code was already written.
`optimizer.py`	2	Train function was optimized by implementing `def step` and doing gradient and optimizer in the same step with the help of LLM to reduce runtime.
`plot_colors.py`	2	LLM used to set up `set.style()` function.

Appendix B: Additional Figures and Tables

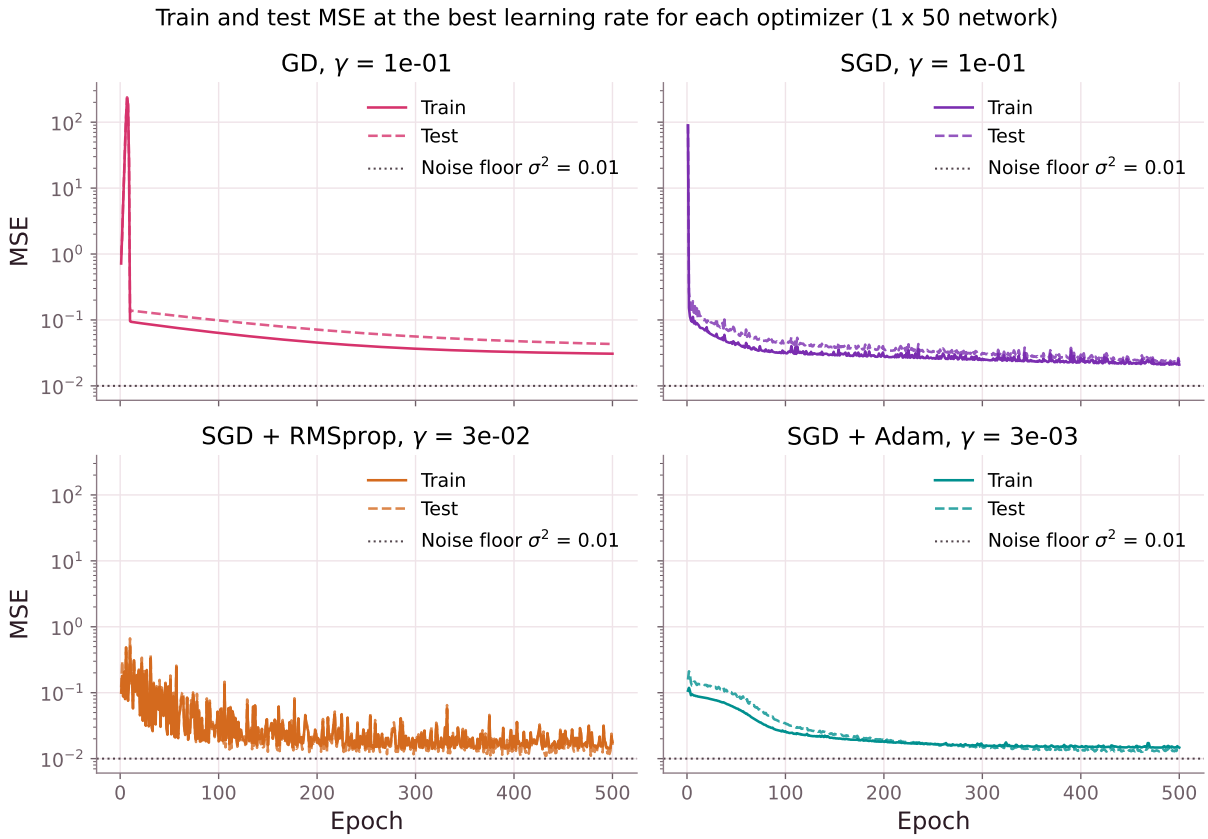


Figure 13: Training (solid) and test (dashed) MSE on the Runge data as a function of epochs, for each gradient method at its best learning rate from Fig. 1: GD and SGD with $\gamma = 10^{-1}$, SGD + RMSprop with $\gamma = 3.2 \cdot 10^{-2}$ and SGD + Adam with $\gamma = 3.2 \cdot 10^{-3}$. All panels use a 1×50 sigmoid network, minibatches of 16 for the stochastic methods, and the same initial weights (seed 2026). The dotted line is the noise floor $\sigma^2 = 0.01$. The panels share axes, and the MSE is shown on a logarithmic scale. SGD + Adam converges smoothly to near the noise floor, while plain GD and SGD are still far above it after 500 epochs.

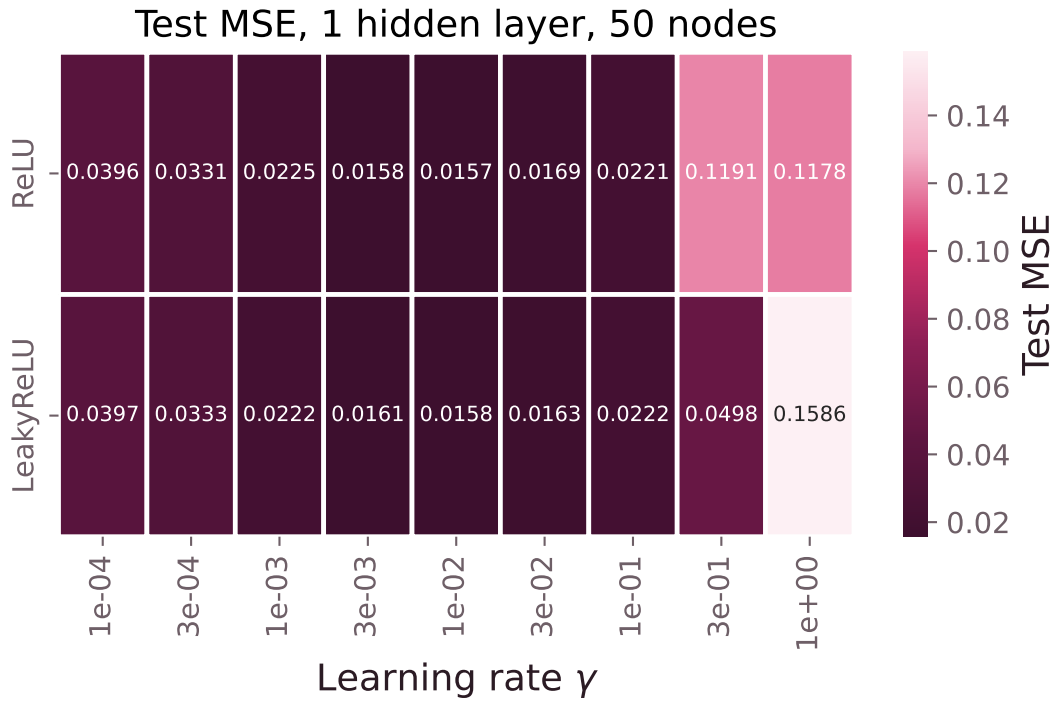


Figure 14: The best learning rate for ReLU and LeakyReLU are 0.01 with a test MSE of 0.0157 and 0.0158 respectively

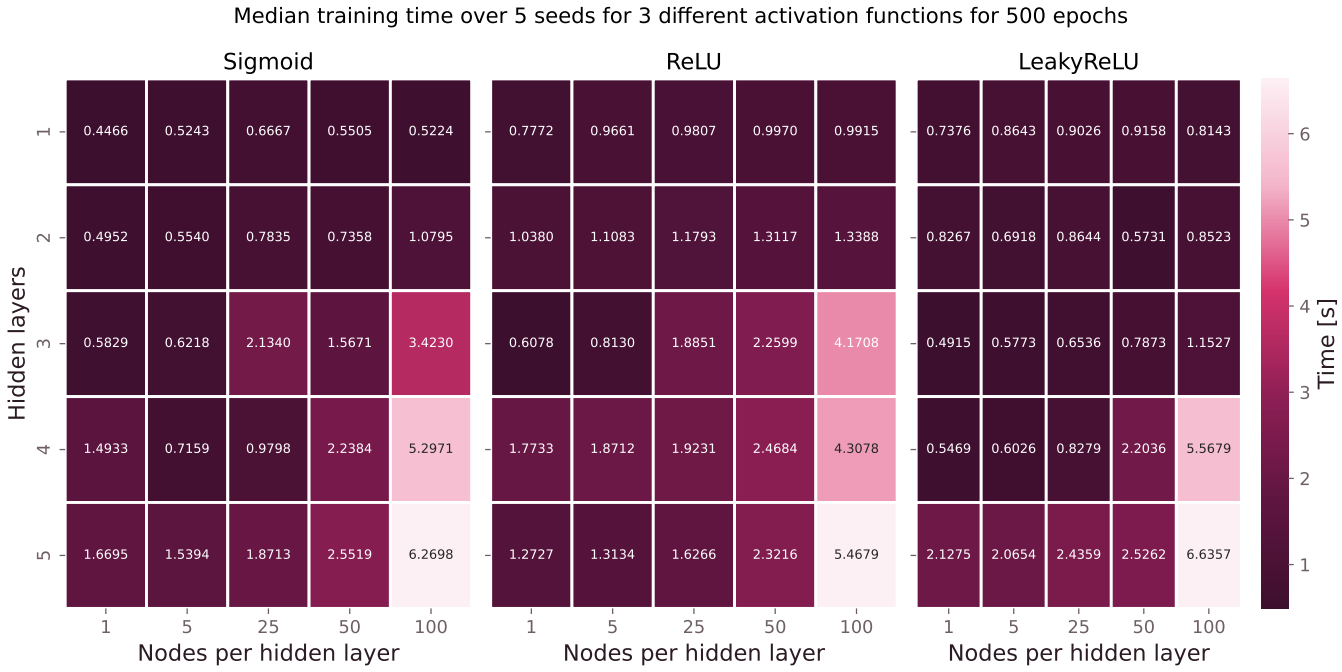


Figure 15: Increasing the number of hidden layers and the number of nodes increases the computation time for all activation functions.

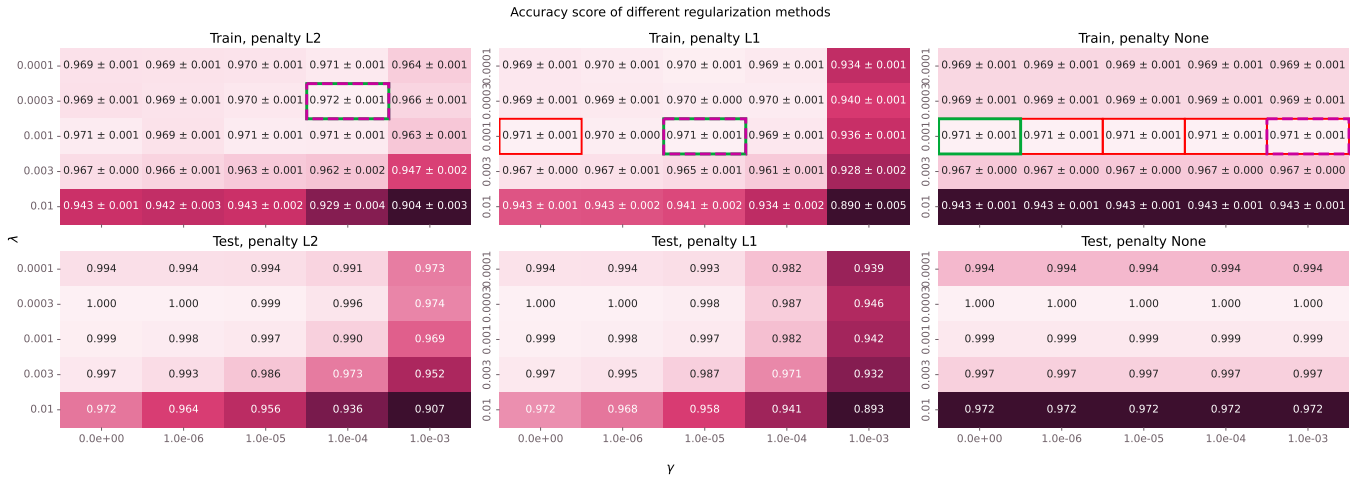


Figure 16: Coarse grid accuracy heatmap over parameter grid with γ = [1e-4, 3e-4, 1e-3, 3e-3, 1e-2], λ [0, 1e-6, 1e-5, 1e-4, 1e-3] and penalties = ['L2', 'L1', None]. 50 epochs, with one hidden layer of 50 nodes with ReLU and softmax output.

Table VI: Classification report results for Logistic regression (LG) model as marked in table VIII with CM and ROC AUC as in fig. 9a. Per-class precision, recall and F1-score on the test set. Support is the number of test samples per class.

Class	Precision	Recall	F1-score	Support
0	0.954	0.979	0.966	980
1	0.969	0.981	0.975	1135
2	0.932	0.905	0.918	1032
3	0.907	0.917	0.912	1010
4	0.936	0.933	0.934	982
5	0.908	0.871	0.889	892
6	0.943	0.952	0.948	958
7	0.925	0.926	0.926	1028
8	0.879	0.894	0.887	974
9	0.918	0.911	0.914	1009
Accuracy		0.928		10000
Macro avg.	0.927	0.927	0.927	10000
Weighted avg.	0.928	0.928	0.928	10000



Figure 17: Training history logged every 5th evaluation of select models. The best model's cost reduces and converges, but remains high compared to the best model with no penalty.

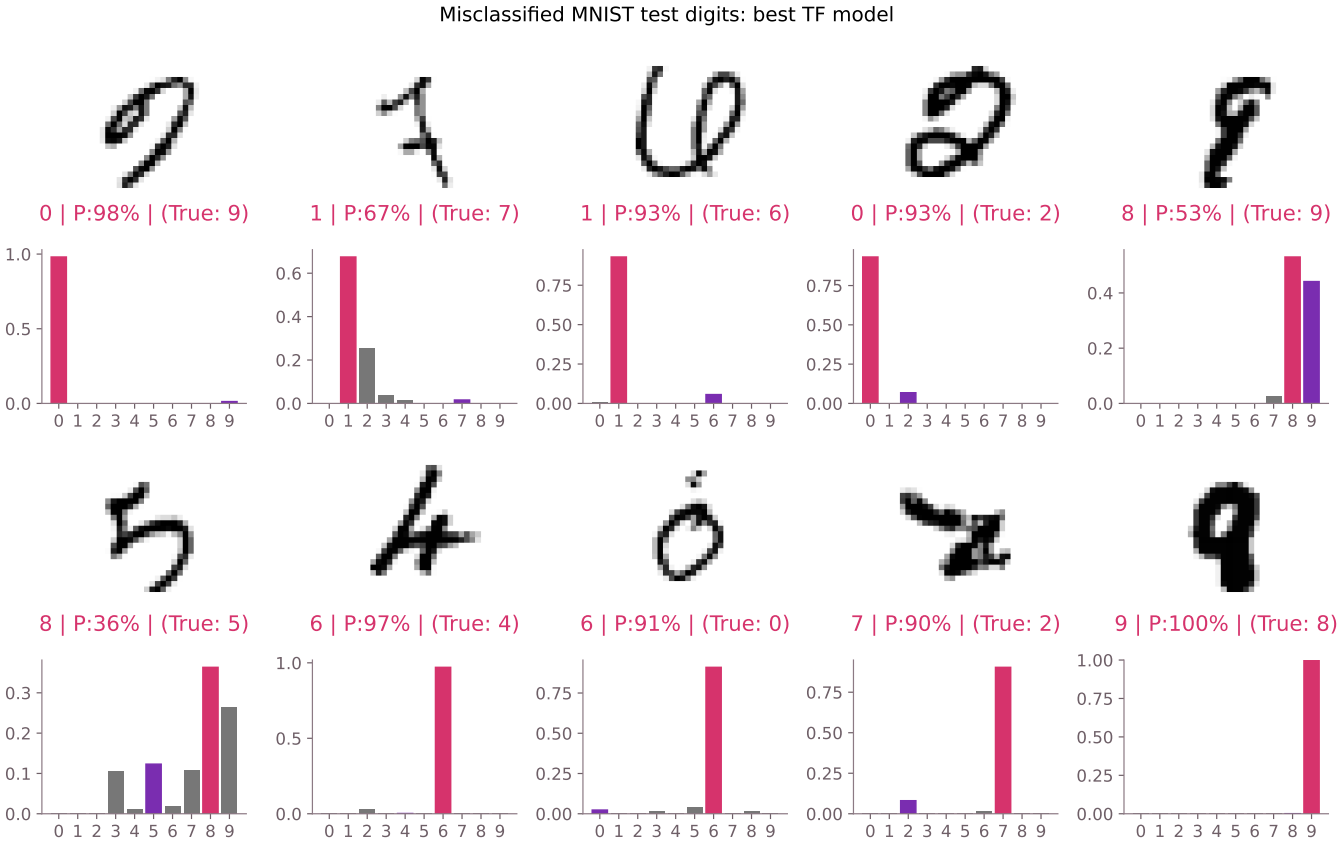


Figure 18: With the TFBM model, digits where the prediction was wrong were picked out. In several of the cases, determining the correct digit with the human eye is challenging.

Table VII: Per-class precision (P), recall (R) and F1-score for the four models on the test set. Support is the number of test samples per class. Figure 11 displays the difference visually, with accompanying models and their parameters in table VIII.

	OO: Cheapest			OO: Best			TF: Cheapest			TF: Best			
Class	P	R	F1	P	R	F1	P	R	F1	P	R	F1	Support
0	0.98	0.987	0.983	0.979	0.992	0.985	0.979	0.993	0.986	0.979	0.993	0.986	980
1	0.982	0.992	0.987	0.989	0.992	0.991	0.99	0.996	0.993	0.99	0.996	0.993	1135
2	0.98	0.974	0.977	0.99	0.953	0.971	0.988	0.977	0.982	0.988	0.977	0.982	1032
3	0.972	0.983	0.977	0.979	0.977	0.978	0.97	0.985	0.977	0.97	0.985	0.977	1010
4	0.967	0.979	0.973	0.978	0.984	0.981	0.982	0.977	0.979	0.982	0.977	0.979	982
5	0.975	0.979	0.977	0.981	0.979	0.98	0.987	0.97	0.979	0.987	0.97	0.979	892
6	0.988	0.967	0.977	0.972	0.989	0.98	0.983	0.986	0.985	0.983	0.986	0.985	958
7	0.977	0.974	0.975	0.989	0.972	0.98	0.981	0.982	0.982	0.981	0.982	0.982	1028
8	0.972	0.967	0.97	0.948	0.98	0.964	0.982	0.971	0.977	0.982	0.971	0.977	974
9	0.971	0.96	0.966	0.982	0.971	0.977	0.97	0.973	0.972	0.97	0.973	0.972	1009
accuracy		0.976			0.979			0.981			0.981		10000
macro avg.	0.976	0.976	0.976	0.979	0.979	0.979	0.981	0.981	0.981	0.981	0.981	0.981	10000
weighted avg.	0.976	0.976	0.976	0.979	0.979	0.979	0.981	0.981	0.981	0.981	0.981	0.981	10000

Table VIII: Clustered model configurations shown in fig. 9b. Shading: Cluster 1; Cluster 2; Cluster 3; Cluster 4. Fixed parameters: 150 epochs with minibatch size of 10, softmax output and cost being multiclass cross entropy. Scores and fit times are mean ± standard deviation.

Train score	Test score	Fit time	Nodes per layer	Penalty type	Penalty	Learning rate	Plot marker
0.939 ± 0.001	0.925 ± 0.001	161.1 ± 20.1	[10]	L1	0	1e-04	Logistic regression
0.939 ± 0.001	0.925 ± 0.001	149.1 ± 8.1	[10]	L2	0	1e-04	–
0.937 ± 0.001	0.925 ± 0.001	141.7 ± 18.7	[10]	L1	1e-05	1e-04	–
0.937 ± 0.001	0.924 ± 0.001	145.9 ± 20.2	[10]	L2	1e-05	1e-04	–
0.929 ± 0.001	0.921 ± 0.002	141.1 ± 20.6	[10]	L1	1e-04	1e-04	–
0.932 ± 0.001	0.923 ± 0.002	134.0 ± 22.0	[10]	L2	1e-04	1e-04	–
0.942 ± 0.001	0.921 ± 0.002	137.9 ± 27.8	[10]	L1	0	3e-04	–
0.942 ± 0.001	0.921 ± 0.002	137.9 ± 18.3	[10]	L2	0	3e-04	–
0.939 ± 0.001	0.922 ± 0.002	146.6 ± 20.3	[10]	L1	1e-05	3e-04	–
0.939 ± 0.001	0.922 ± 0.002	133.4 ± 14.3	[10]	L2	1e-05	3e-04	–
0.929 ± 0.001	0.920 ± 0.002	139.8 ± 23.4	[10]	L1	1e-04	3e-04	–
0.932 ± 0.001	0.921 ± 0.002	133.1 ± 16.7	[10]	L2	1e-04	3e-04	–
0.941 ± 0.001	0.914 ± 0.002	140.1 ± 20.1	[10]	L1	0	1e-03	–
0.941 ± 0.001	0.914 ± 0.002	143.0 ± 38.7	[10]	L2	0	1e-03	–
0.938 ± 0.001	0.917 ± 0.003	153.6 ± 23.6	[10]	L1	1e-05	1e-03	–
0.936 ± 0.001	0.918 ± 0.003	141.5 ± 20.4	[10]	L2	1e-05	1e-03	–
0.927 ± 0.001	0.917 ± 0.003	152.0 ± 28.2	[10]	L1	1e-04	1e-03	–
0.929 ± 0.001	0.918 ± 0.003	148.4 ± 42.9	[10]	L2	1e-04	1e-03	–
0.939 ± 0.001	0.925 ± 0.001	146.6 ± 13.5	[10]	nan	nan	1e-04	–
0.942 ± 0.001	0.921 ± 0.002	146.1 ± 10.8	[10]	nan	nan	3e-04	–
0.941 ± 0.001	0.914 ± 0.002	155.5 ± 4.4	[10]	nan	nan	1e-03	–
1.000 ± 0.000	0.968 ± 0.002	418.8 ± 77.8	[50, 10]	L1	0	1e-04	–
1.000 ± 0.000	0.968 ± 0.002	401.1 ± 86.3	[50, 10]	L2	0	1e-04	–
1.000 ± 0.000	0.971 ± 0.002	489.3 ± 113.0	[50, 10]	L1	1e-05	1e-04	–
1.000 ± 0.000	0.969 ± 0.002	509.7 ± 120.6	[50, 10]	L2	1e-05	1e-04	–
0.991 ± 0.000	0.971 ± 0.002	555.8 ± 125.0	[50, 10]	L1	1e-04	1e-04	–
0.998 ± 0.000	0.973 ± 0.002	392.5 ± 87.1	[50, 10]	L2	1e-04	1e-04	–
1.000 ± 0.000	0.968 ± 0.002	384.0 ± 70.8	[50, 10]	L1	0	3e-04	–
1.000 ± 0.000	0.968 ± 0.002	411.0 ± 58.8	[50, 10]	L2	0	3e-04	–
0.999 ± 0.001	0.969 ± 0.003	438.3 ± 61.3	[50, 10]	L1	1e-05	3e-04	–
1.000 ± 0.000	0.971 ± 0.002	500.7 ± 62.2	[50, 10]	L2	1e-05	3e-04	–
0.991 ± 0.001	0.971 ± 0.002	417.5 ± 33.9	[50, 10]	L1	1e-04	3e-04	–
0.998 ± 0.000	0.975 ± 0.001	454.2 ± 58.5	[50, 10]	L2	1e-04	3e-04	–
0.999 ± 0.001	0.970 ± 0.002	503.5 ± 74.5	[50, 10]	L1	0	1e-03	–
0.999 ± 0.001	0.970 ± 0.002	383.5 ± 98.1	[50, 10]	L2	0	1e-03	–
0.997 ± 0.001	0.971 ± 0.002	410.4 ± 85.6	[50, 10]	L1	1e-05	1e-03	–
0.997 ± 0.001	0.971 ± 0.002	505.1 ± 57.7	[50, 10]	L2	1e-05	1e-03	–
0.985 ± 0.001	0.971 ± 0.001	474.5 ± 40.7	[50, 10]	L1	1e-04	1e-03	–
0.992 ± 0.001	0.974 ± 0.002	404.6 ± 73.7	[50, 10]	L2	1e-04	1e-03	–
1.000 ± 0.001	0.966 ± 0.002	457.3 ± 83.3	[50, 50, 10]	L1	0	1e-04	–
1.000 ± 0.001	0.966 ± 0.002	518.3 ± 71.1	[50, 50, 10]	L2	0	1e-04	–
1.000 ± 0.000	0.970 ± 0.001	506.4 ± 55.2	[50, 50, 10]	L1	1e-05	1e-04	–
1.000 ± 0.001	0.969 ± 0.001	500.6 ± 64.5	[50, 50, 10]	L2	1e-05	1e-04	–
0.995 ± 0.000	0.971 ± 0.002	531.2 ± 68.6	[50, 50, 10]	L1	1e-04	1e-04	–
1.000 ± 0.000	0.972 ± 0.002	480.7 ± 62.4	[50, 50, 10]	L2	1e-04	1e-04	–
0.999 ± 0.001	0.970 ± 0.001	482.4 ± 100.1	[50, 50, 10]	L1	0	3e-04	–
0.999 ± 0.001	0.970 ± 0.001	467.7 ± 78.4	[50, 50, 10]	L2	0	3e-04	–
0.999 ± 0.000	0.973 ± 0.001	576.9 ± 70.2	[50, 50, 10]	L1	1e-05	3e-04	–
0.999 ± 0.001	0.972 ± 0.002	529.6 ± 73.9	[50, 50, 10]	L2	1e-05	3e-04	–
0.993 ± 0.001	0.973 ± 0.002	507.5 ± 52.2	[50, 50, 10]	L1	1e-04	3e-04	–
0.997 ± 0.002	0.974 ± 0.002	475.6 ± 19.4	[50, 50, 10]	L2	1e-04	3e-04	–
0.998 ± 0.001	0.972 ± 0.002	485.7 ± 17.8	[50, 50, 10]	L1	0	1e-03	–
0.998 ± 0.001	0.972 ± 0.002	498.9 ± 21.2	[50, 50, 10]	L2	0	1e-03	–
0.997 ± 0.001	0.972 ± 0.002	542.0 ± 18.9	[50, 50, 10]	L1	1e-05	1e-03	–
0.997 ± 0.000	0.973 ± 0.001	519.1 ± 34.8	[50, 50, 10]	L2	1e-05	1e-03	–

Continued on next page

Train score	Test score	Fit time	Nodes per layer	Penalty type	Penalty	Learning rate	Plot marker
0.986 ± 0.002	0.971 ± 0.002	542.5 ± 20.3	[50, 50, 10]	L1	1e-04	1e-03	–
0.990 ± 0.003	0.972 ± 0.002	573.8 ± 24.1	[50, 50, 10]	L2	1e-04	1e-03	–
1.000 ± 0.000	0.968 ± 0.002	358.9 ± 5.6	[50, 10]	nan	nan	1e-04	–
1.000 ± 0.000	0.968 ± 0.002	363.5 ± 10.3	[50, 10]	nan	nan	3e-04	–
0.999 ± 0.001	0.970 ± 0.002	407.1 ± 27.3	[50, 10]	nan	nan	1e-03	–
1.000 ± 0.001	0.966 ± 0.002	471.8 ± 19.1	[50, 50, 10]	nan	nan	1e-04	–
0.999 ± 0.001	0.970 ± 0.001	459.9 ± 27.5	[50, 50, 10]	nan	nan	3e-04	–
0.998 ± 0.001	0.972 ± 0.002	505.6 ± 17.6	[50, 50, 10]	nan	nan	1e-03	–
0.999 ± 0.000	0.976 ± 0.001	594.8 ± 13.5	[100, 50, 25, 10]	nan	nan	1e-03	Best cluster 2
0.997 ± 0.002	0.977 ± 0.001	858.8 ± 47.8	[100, 50, 25, 10]	L2	1e-05	1e-03	–
0.986 ± 0.002	0.972 ± 0.003	827.9 ± 48.8	[100, 50, 25, 10]	L1	1e-04	1e-03	–
0.991 ± 0.001	0.974 ± 0.003	778.7 ± 37.4	[100, 50, 25, 10]	L2	1e-04	1e-03	–
1.000 ± 0.000	0.976 ± 0.002	822.6 ± 21.6	[100, 50, 25, 10]	nan	nan	3e-04	–
1.000 ± 0.000	0.975 ± 0.001	985.9 ± 56.1	[100, 50, 10]	L1	0	1e-04	–
1.000 ± 0.000	0.975 ± 0.001	1001.9 ± 132.2	[100, 50, 10]	L2	0	1e-04	–
1.000 ± 0.000	0.976 ± 0.001	1072.9 ± 37.9	[100, 50, 10]	L1	1e-05	1e-04	–
1.000 ± 0.000	0.977 ± 0.001	1056.2 ± 34.4	[100, 50, 10]	L2	1e-05	1e-04	–
0.997 ± 0.001	0.975 ± 0.001	1058.1 ± 78.5	[100, 50, 10]	L1	1e-04	1e-04	–
1.000 ± 0.000	0.978 ± 0.002	1060.4 ± 77.4	[100, 50, 10]	L2	1e-04	1e-04	Best cheapest
1.000 ± 0.000	0.976 ± 0.001	1128.5 ± 34.8	[100, 50, 10]	L1	0	3e-04	–
1.000 ± 0.000	0.976 ± 0.001	1052.4 ± 55.7	[100, 50, 10]	L2	0	3e-04	–
0.999 ± 0.000	0.978 ± 0.001	1188.3 ± 60.6	[100, 50, 10]	L1	1e-05	3e-04	–
0.999 ± 0.000	0.977 ± 0.001	1193.7 ± 55.5	[100, 50, 10]	L2	1e-05	3e-04	–
0.993 ± 0.001	0.974 ± 0.003	1100.6 ± 64.8	[100, 50, 10]	L1	1e-04	3e-04	–
0.998 ± 0.000	0.978 ± 0.002	1125.7 ± 119.1	[100, 50, 10]	L2	1e-04	3e-04	Best
0.999 ± 0.000	0.977 ± 0.002	1140.5 ± 91.3	[100, 50, 10]	L1	0	1e-03	–
0.999 ± 0.000	0.977 ± 0.002	1080.9 ± 125.7	[100, 50, 10]	L2	0	1e-03	–
0.997 ± 0.001	0.975 ± 0.002	1250.6 ± 32.6	[100, 50, 10]	L1	1e-05	1e-03	–
0.997 ± 0.001	0.976 ± 0.001	1044.4 ± 114.4	[100, 50, 10]	L2	1e-05	1e-03	–
0.986 ± 0.001	0.972 ± 0.003	1103.3 ± 91.9	[100, 50, 10]	L1	1e-04	1e-03	–
0.991 ± 0.002	0.974 ± 0.002	1109.8 ± 163.2	[100, 50, 10]	L2	1e-04	1e-03	–
1.000 ± 0.001	0.973 ± 0.001	1297.8 ± 32.9	[100, 50, 25, 10]	L1	0	1e-04	–
1.000 ± 0.001	0.973 ± 0.001	1209.2 ± 131.9	[100, 50, 25, 10]	L2	0	1e-04	–
1.000 ± 0.000	0.973 ± 0.001	1089.4 ± 163.9	[100, 50, 25, 10]	L1	1e-05	1e-04	–
1.000 ± 0.000	0.975 ± 0.002	1148.8 ± 104.4	[100, 50, 25, 10]	L2	1e-05	1e-04	–
0.998 ± 0.000	0.973 ± 0.002	1121.6 ± 104.6	[100, 50, 25, 10]	L1	1e-04	1e-04	–
1.000 ± 0.000	0.976 ± 0.002	1209.8 ± 116.6	[100, 50, 25, 10]	L2	1e-04	1e-04	–
1.000 ± 0.000	0.976 ± 0.002	1235.4 ± 48.5	[100, 50, 25, 10]	L1	0	3e-04	–
1.000 ± 0.000	0.976 ± 0.002	1029.4 ± 103.3	[100, 50, 25, 10]	L2	0	3e-04	–
0.999 ± 0.001	0.976 ± 0.002	1113.9 ± 99.9	[100, 50, 25, 10]	L1	1e-05	3e-04	–
1.000 ± 0.000	0.977 ± 0.002	1092.2 ± 69.1	[100, 50, 25, 10]	L2	1e-05	3e-04	–
0.994 ± 0.001	0.973 ± 0.002	1162.2 ± 46.4	[100, 50, 25, 10]	L1	1e-04	3e-04	–
0.998 ± 0.001	0.978 ± 0.003	1076.7 ± 118.8	[100, 50, 25, 10]	L2	1e-04	3e-04	–
0.999 ± 0.000	0.976 ± 0.001	1134.8 ± 56.1	[100, 50, 25, 10]	L1	0	1e-03	–
0.999 ± 0.000	0.976 ± 0.001	1015.1 ± 70.2	[100, 50, 25, 10]	L2	0	1e-03	–
0.996 ± 0.001	0.976 ± 0.001	1047.7 ± 44.3	[100, 50, 25, 10]	L1	1e-05	1e-03	–
1.000 ± 0.000	0.975 ± 0.001	916.5 ± 28.7	[100, 50, 10]	nan	nan	1e-04	–
1.000 ± 0.000	0.976 ± 0.001	974.5 ± 68.7	[100, 50, 10]	nan	nan	3e-04	–
0.999 ± 0.000	0.977 ± 0.002	921.4 ± 41.9	[100, 50, 10]	nan	nan	1e-03	–
1.000 ± 0.001	0.973 ± 0.001	972.3 ± 12.5	[100, 50, 25, 10]	nan	nan	1e-04	–